

Classical ML vs. LLM for Credit Scoring

June Progress Report

Summary

- No prompt-engineering trick reliably beat the simple base prompt. Chain-of-thought and forced high reasoning both looked good on a first run, then fell back under repeat testing.
- The LLM's confidence is poorly calibrated (ECE around 0.20). It is right or wrong with almost the same confidence, so a decision threshold on its probability is not worth tuning, and XGBoost keeps the AUC edge.
- Each experimental phase carries its own cost and explainability analysis, reported inside the phase rather than in a separate block, matching the repo structure.
- The final 1,000-loan test is built but not yet run, held for your decision on scope.

1. Model selection (Phase 1)

Key findings

- We stayed with GPT-5.4. It is the strongest model on the task, and also the one that supports prompt caching cleanly, answers fast, and returns token log-probabilities for the calibration work.
- Description flip: for GPT-5.4, removing the borrower description improves results (81% no-desc vs 78% with-desc), the opposite of April. The no-desc setting is both the fairest comparison against XGBoost and the best one.
- Reasoning effort: forcing higher reasoning cost 2 to 2.6 times more and did not help. A consistency check shows high effort drifts slightly below the native default once repeated.
- Log-probabilities (the feature we discussed last meeting), tested on GPT-5.4 and Gemini 2.5 Pro: the model is confidently right or confidently wrong, so threshold tuning is not viable.

1.1 Model comparison

Table 1. Five models on the 100-loan tuning sample (15 defaults), best condition shown. Best value per column in bold.

Model (best condition)	Acc.	CO Prec.	CO Rec.	CO F1	AUC
GPT-5.4 (no-desc)	0.81	0.375	0.40	0.387	0.624
Gemini 2.5 Pro (with-desc)	0.64	0.256	0.733	0.379	0.680
Claude Opus 4.8 (with-desc)	0.69	0.233	0.467	0.311	n/a
Gemini 3.5 Flash (with-desc)	0.64	0.216	0.533	0.308	n/a
Claude Sonnet 4.6 (no-desc)	0.515	0.200	0.733	0.314	n/a
XGBoost (baseline, same sample)	0.65	0.206	0.467	0.286	0.671

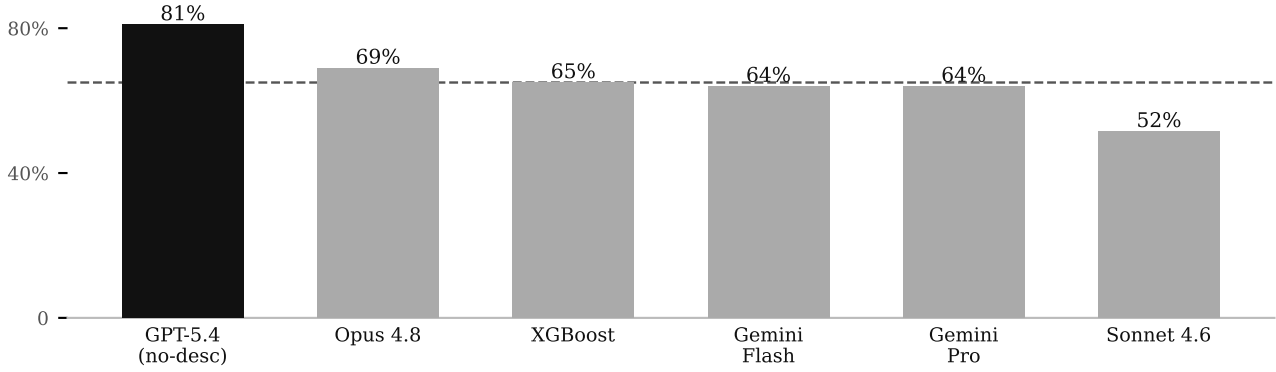


Figure 1. Accuracy by model on the same 100-loan sample; dashed line is the XGBoost baseline.

GPT-5.4 in the no-description condition is the only model to clear the XGBoost baseline on both accuracy and Charged-Off F1. Gemini 2.5 Pro reaches a comparable F1 (0.379) and the best AUC (0.680), but only with the description and at 64% accuracy. The larger and more expensive Claude models do not win here: Opus lands at 67 to 69% and Sonnet near 51%.

1.2 Reasoning effort

Table 2. GPT-5.4 reasoning_effort sweep, no-desc, single run per setting.

Setting	Acc.	CO F1	Out tokens	Cost / 100
Omitted (native default)	0.81	0.387	84	€0.25
low	0.74	0.350	117	€0.30
medium	0.76	0.294	237	€0.48
high	0.80	0.375	355	€0.66

The native default (no reasoning_effort set) gave the best accuracy at the lowest cost. The single high-effort run looked competitive (80% / 0.375), so we repeated it three times to see whether that held.

Table 2b. High-effort stability, three repeat runs on the same sample.

Run	Accuracy	CO F1
1	0.76	0.333
2	0.76	0.333
3	0.74	0.235
Mean	0.753	0.300

It did not hold. Averaged over three runs, high effort sits at 75.3% / 0.30, below the native default, and its strong first run was a lucky draw. This is the same pattern we see with chain-of-thought in Phase 2: one good run that regresses on repetition. Forcing the model to write long reasoning over clean numeric tables adds cost and variance without adding accuracy.

1.3 Confidence and calibration

Table 3. Calibration on logged predictions. Conf. gap = mean confidence when correct minus when wrong (near zero means confidence does not separate right from wrong answers).

Model	n	Acc.	Brier	ECE	Conf. gap
GPT-5.4	800	0.788	0.201	0.204	0.032
Gemini 2.5 Pro	203	0.601	0.399	0.400	-0.001

GPT-5.4 is systematically over-confident (ECE around 0.20), and its confidence barely separates correct from incorrect calls (a 0.03 gap). Gemini Pro shows no separation at all. Two things

follow. First, tuning a decision threshold on the LLM's confidence is not worthwhile, because the confidence carries almost no information about whether the answer is right. Second, although GPT-5.4 wins on accuracy and F1, XGBoost still produces better-ranked probabilities (AUC 0.671 vs 0.624). The honest framing: on this task the LLM is the better classifier but the worse probability estimator.

1.4 Cost

Table 4. Logged cost per 100 loans, by model.

Model	Cost / 100 loans
Gemini 2.5 Pro	€0.15
Gemini 3.5 Flash	€0.16
GPT-5.4	€0.36
Claude Sonnet 4.6	€0.37
Claude Opus 4.8	€0.64

Per-decision cost is small and now fully logged for every call (total experimental spend to date is roughly €14 across about 4,000 calls). GPT-5.4 is mid-priced, and with prompt caching on the repeated instruction block the marginal cost of the winning configuration drops further. At these levels the API bill is a rounding error next to the credit outcomes the decision drives, a point Phase 2 makes concrete in euros.

1.5 Explainability: GPT-5.4 rationale vs XGBoost SHAP

This is the first half of the brief's explainability comparison: what does each model actually base its decision on?

Table 5. XGBoost, top features by mean |SHAP| value.

Rank	Feature	Mean SHAP
1	Interest rate	0.336
2	Term	0.224
3	Annual income	0.202
4	Accounts opened, last 24m	0.162
5	FICO (low)	0.090
6	Home ownership	0.088
7	Loan amount	0.087
8	Debt-to-income	0.081

For the LLM side we used a GPT-5.4 judge to summarise GPT-5.4's own reasoning across the sample. The fingerprint is consistent: it anchors on FICO band, recent delinquencies, bankruptcies and public records, revolving utilisation, DTI and credit-history length, then on loan grade, interest rate, income verification and employment length. Its posture is moderately conservative; it approves when the core credit file looks clean and leans on grade and pricing as a risk summary when negatives stack up.

Quantitatively the two feature-reliance rankings correlate only weakly: Spearman rho is about 0.20 across the full feature set (from notebook 01g), so at the level of the whole file the two models prioritise features quite differently. XGBoost leans on term and on recent account openings, which the LLM rarely leads with, while the LLM foregrounds delinquency and derogatory history. What they share is the handful of dominant signals (interest rate and grade, FICO, DTI), and those drive most decisions. That reconciles the weak ranking correlation with the high error overlap in Phase 3: the two models agree on the few features that decide the hard cases, even if they disagree on the long tail. It is also reassuring for explainability, since the LLM anchors on the same core credit signals an analyst would, not on spurious ones.

2. Prompt engineering (Phase 2)

An earlier version of this phase was shown last meeting on Llama-3.3-70b. The results below are a fresh run on GPT-5.4 and replace it; the qualitative conclusions are similar but the numbers differ.

Key findings

- No engineered prompt reliably beat the base prompt. Chain-of-thought topped the first run (83% / 0.370) but regressed under consistency testing to a mean of 0.312 Charged-Off F1, below the base prompt's 0.327.
- The base prompt stays the choice: best average Charged-Off F1, simplest, cheapest.
- Batching loans into one shared context window collapses accuracy (81% to 62%) through an anchoring effect, and in euros it destroys far more value than the token saving is worth.
- Compact (TOON) formatting is nearly free in accuracy and about 18% cheaper, but prompt caching achieves the same saving without the accuracy cost.

2.1 Prompt variants

Table 6. Seven system-prompt variants, GPT-5.4, no-desc, single comparison run.

Variant	Acc.	CO F1	AUC
Base prompt (reference, Phase 1)	0.81	0.387	0.624
chain_of_thought	0.83	0.370	0.645
few_shot (4 examples)	0.82	0.357	0.637
structured_4factor	0.78	0.353	0.613
risk_signal_guide	0.75	0.324	0.635
top_features_only (8 features)	0.68	0.304	0.632
conservative	0.54	0.303	0.620

Chain-of-thought posted the best single-run accuracy and AUC, so we carried it into consistency testing. It then failed to hold.

Table 7. Chain-of-thought vs base prompt, three repeat runs each (mean).

Prompt	Mean accuracy	Mean CO F1
Base prompt	0.807	0.327
chain_of_thought	0.810	0.312

Chain-of-thought's first run (0.370) fell to 0.296 on the second and third. Averaged, it is below the base prompt on Charged-Off F1, the metric that matters for the minority class. We therefore treat no variant as an improvement over the base prompt. Carried onto the held-out batch, chain-of-thought dropped to 0.087, in line with the fragility of minority-class F1 on small samples (Section 4).

2.2 The formatting and batching tax

Table 8. Four input designs, GPT-5.4, mean of three runs.

Condition	Acc.	CO Rec.	CO F1	Cost / 100
Natural language, individual	0.807	0.311	0.327	€0.249
TOON (compact), individual	0.783	0.333	0.316	€0.205
Natural language, batched	0.620	0.667	0.345	€0.173
Batched TOON	0.620	0.400	0.237	€0.098

Compact formatting costs about 2.4 points of accuracy for an 18% token saving, almost free. Batching is different. Scoring all 100 loans in one shared context window collapses accuracy from

81% to 62%. Default recall rises, because the model turns hyper-vigilant after seeing several high-risk files earlier in the same window, but precision craters and it rejects many good borrowers. A borrower's decision then depends on which other applications happened to share the batch, an objectivity problem on top of the accuracy loss.

2.3 Financial analysis (euros)

Token cost alone understates what these design choices are worth. We simulate a 1,000-loan portfolio (€10k average loan, 15% default rate, 50% loss-given-default, €2,000 expected interest profit per repaid loan) and price each design by its credit outcomes, not just its API bill.

Table 9. Net portfolio cost vs the control design, per 1,000 loans (illustrative assumptions).

Condition	API token cost	Net cost vs control
Natural, individual	€2.49	baseline
TOON, individual	€2.05	+€39k
Batching	€1.73	+€216k
Batched TOON	€0.98	+€345k

A 60% token saving from batching destroys roughly €345k of value per 1,000 loans through false rejections and missed defaults. For a production design the conclusion is to optimise accuracy first, score loans individually, and recover cost through prompt caching rather than through lossy formatting or batching.

2.4 Explainability: why the variants failed

The prompt fingerprints (the 02c judge analysis) line up cleanly with the metrics, which is the useful part: the prompts did what they said, the behaviour just was not helpful.

Table 10. What each variant pushed the model toward, and the measured effect.

Variant	What the prompt pushed	Measured effect
conservative	flag defaults when unsure	recall 0.67, precision 0.20, acc 54%
chain_of_thought	long step-by-step reasoning	strong first run, high variance, regresses
top_features_only	only 8 top features	information loss, accuracy 68%
few_shot / 4factor / guide	more instruction, structure	more risk-flagging, none beats base F1

The pattern is consistent. The conservative variant is the clearest case: an instruction to lean toward flagging defaults when uncertain produces exactly the precision-recall trade you would predict, catching more defaults but drowning them in false alarms, so accuracy falls to 54%. The variants that add an explicit framework (structured 4factor, risk signal guide) or extra reasoning (chain_of_thought) all nudge the model to weight stacked negatives more heavily, which raises recall a little but costs precision or adds variance. top_features_only simply removes information and loses accuracy. The base prompt does best because GPT-5.4's untouched default is already the most balanced reasoner in the 01f fingerprints; heavier prompting overrides that balance rather than improving it. On this task the model's native judgment is hard to beat by telling it how to think.

3. Hybrid ML + LLM (Phase 3)

Key findings

- Our Phase 1 error analysis already found that about 84% of the loans one model gets wrong are also wrong for the other, so we did not expect a hybrid to help much.
- A soft-probability blend marginally tops the leaderboard, but the gain over the LLM alone is small and unstable across samples, consistent with that overlap.
- The test set is strictly held out here: parameters are tuned on the tuning sample and the strategy is selected on the robustness batch only.

The 84% error overlap is the key context. Section 1.5 already showed why: XGBoost and the LLM lean on the same dominant signals (interest rate and grade, FICO, DTI), so when a loan fools one it usually fools the other. A blend can only add value where the two models disagree, and they rarely do on the hard cases.

Table 11. Hybrid strategies, Charged-Off F1 on two samples (test set excluded).

Strategy	Tuning sample	Robustness batch
Soft blend (binary)	0.400	0.133
Intersection (XGB AND LLM)	0.357	0.105
LLM alone	0.387	0.095
5A risk blend	0.400	0.087
Confidence-gated routing	0.400	0.057
XGBoost alone	0.311	0.049

The soft blend edges the LLM-alone baseline (0.40 vs 0.387 on tuning; 0.133 vs 0.095 on the robustness batch), but the absolute numbers are low and noisy and no strategy delivers a step change. Confidence-gated routing is interesting on cost grounds, since it sends easy cases to XGBoost for free and only the hard ones to the LLM, but here it did not improve F1, so the saving does not justify the added complexity. On this evidence the hybrid is not a compelling direction over the LLM alone.

4. Final benchmark and limitations

4.1 Phase 4 plan (open question)

Phase 4 is the one part we have deliberately not run yet, and we have not fully fixed its scope. As currently planned it is a single final test of the winning prompt (base prompt, no-desc, native reasoning effort) on the held-out set of about 1,000 LendingClub loans, with a financial analysis using both the simulated economics above and LendingClub's real loan figures, plus an error analysis against XGBoost. We are inclined to drop decision-threshold tuning, since the calibration results (Section 1.3) show the LLM's probability is not informative enough for it to be worthwhile.

The test set has never been loaded or evaluated outside this phase, so it stays a clean one-shot. The open question for you: run it now with the locked finalist, or explore more first (additional prompt variants, few-shot example selection, calibration) before we spend the single clean test.

4.2 Honest caveats

Caveats

- Small-sample minority-class F1 is noisy. The held-out batch has only about 9 defaults, so Charged-Off F1 swings widely there for both models. We lean on accuracy and on multi-run averages.
- The LLM is a poor probability estimator. It is over-confident and loses to XGBoost on AUC, so any deployment story must separate a good classifier from a good calibrated risk score.
- Some sweeps are single-run: low and medium reasoning are one run each; high effort and the prompt variants were repeated.
- Public, de-identified data. We use the public LendingClub dataset as an offline testbed. This is prototyping only, with no production or real-customer-data path.
- Phase 3 configuration to confirm before presenting: the committed hybrid results were produced with GPT-5.4 signals rather than the Llama/Groq setup the phase was originally designed around.